基于ArceOS **dev分支**（commit **16096568f5a**），分析多核并发方面的问题。

计划（四步）

1. 在ArceOS和Linux6.12.37之间，进行多核并发相关**代码的对比分析**，针对以下方面：SpinLock、Mutex、WaitQ(Completion)、Resched(Schedule)、Affinity&&Migration、IPI。
2. 以**触发潜在并发问题**为主要目的，执行各类多核并发测试；并自此基础上，**定位**问题的原因。
3. 综合发现的问题，提出修复的PR。
4. 回归测试和更大的压力测试，确认问题的修复。

代码对比

在ArceOS和Linux之间对比实现差异性，分析可能产生的影响。

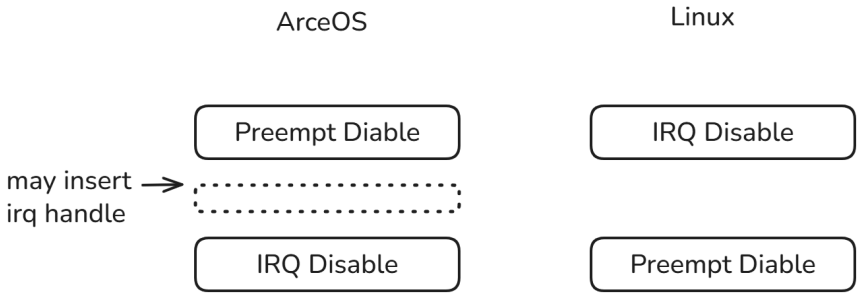
SpinLock

差异：在**加锁**时，ArceOS的顺序是先关抢占、再关中断；而Linux正**相反**，先关中断、再关抢占。

```rust
impl BaseGuard for NoPreemptIrqSave {
    type State = usize;
    fn acquire() -> Self::State {
        // disable preempt
        #[cfg(feature = "preempt")]
        crate_interface::call_interface!(KernelGuardIf::disable_preempt);
        // disable IRQs and save IRQ states
        super::arch::local_irq_save_and_disable()
    }
```

```linux
static inline void __raw_spin_lock_irq(raw_spinlock_t *lock)
{
    local_irq_disable();
    preempt_disable();
    spin_acquire(&lock->dep_map, 0, 0, _RET_IP_);
    LOCK_CONTENDED(lock, do_raw_spin_trylock, do_raw_spin_lock);
}
```

影响：对于ArceOS，Preempt_disable和Irq_disable两行中间可能发生中断，导致加锁过程被意外的拉长，与自旋锁短促应用的场景原则不符；对于Linux，把Preempt_disable放在Irq_disable后面不会产生不良影响，因为两行之间不存在抢占点。相对来说，关中断比关抢占的紧迫性要高，所以Linux的处理方式更合理。



> 此差异不会导致SMP调度死锁问题，不是当前的关键。

Mutex

差异：ArceOS的RawMutex.lock和Linux的mutex_lock，都是分为fast_path和slow_path两层。slow_path方面没有可比性，ArceOS基于第三方的crate *event_listener*实现，那是一个异步协程机制；而Linux是自己的slow_path实现。fast_path方面都是基于对owner_id成员的比较，发现的问题：

ArceOS

```rust
#[inline(always)]
fn lock(&self) {
    let current_id = current().id().as_u64();
    let mut spin = Spin(0);
    let mut owner_id = self.owner_id.load(Ordering::Relaxed);

    loop {
        ... ...

        if spin.spin() {
            owner_id = self.owner_id.load(Ordering::Relaxed);
            continue;
        }

        listener!(self.event => listener);

        owner_id = self.owner_id.load(Ordering::Relaxed);
        if owner_id == 0 {
            continue;
        }

        block_on(listener);
        owner_id = self.owner_id.load(Ordering::Relaxed);
    }
```

Linux

```linux
static struct task_struct *__mutex_trylock_common(struct mutex *lock, bool handoff)
{
    unsigned long owner, curr = (unsigned long)current;

    owner = atomic_long_read(&lock->owner);
    for (;;) { /* must loop, can race against a flag */
        ... ...

        if (atomic_long_try_cmpxchg_acquire(&lock->owner, &owner, task | flags)) {
            ... ...
        }
    }
```

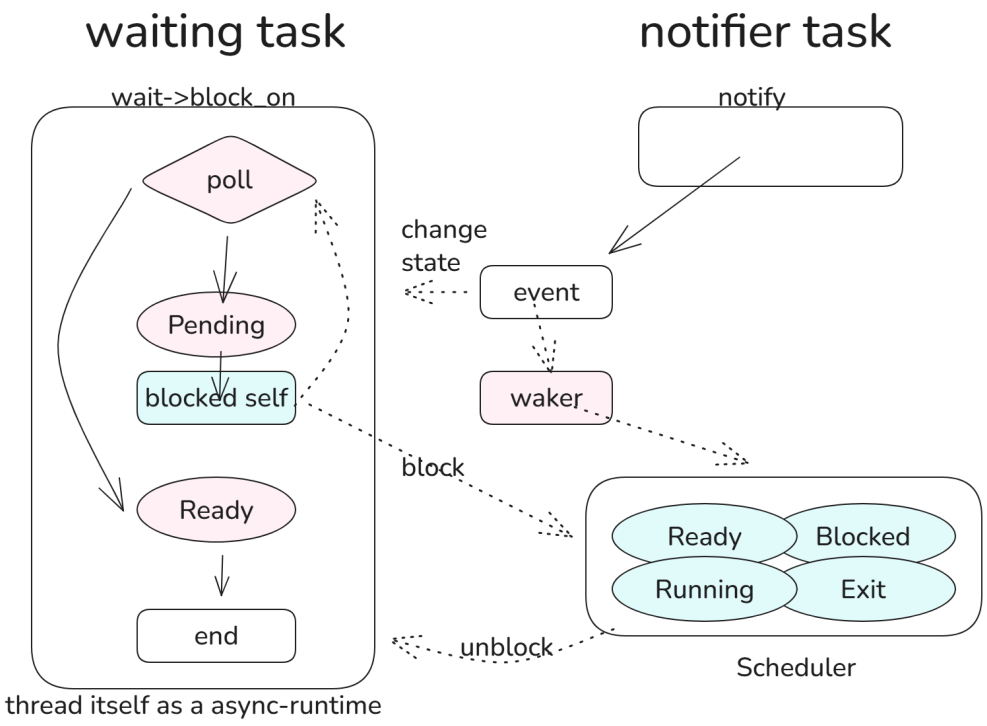
```linux
static noinline void _sched __mutex_unlock_slowpath(struct mutex *lock, unsigned long ip)
{
    ... ...
    owner = atomic_long_read(&lock->owner);
    for (;;) {
        ... ...
        if (atomic_long_try_cmpxchg_release(&lock->owner, &owner, _owner_flags(owner))) {
            if (owner & MUTEX_FLAG_WAITERS)
                break;

            return;
        }
```

在循环中Load原子变量owner_id时，ArceOS用的是Relaxed，Linux使用的是Acquire。

影响：在SMP情况下，lock和unlock可能由不同CPU上的任务执行，ArceOS这里基于Relaxed去访问，可能读到的是本地CPU缓存中的过期值而无法及时解锁，这是一个**“死锁”的隐患**。参照Linux的做法，一般情况下应当优先考虑load+acquire，store+release这样的配合。另外，ArceOS的dev分支基于event_listener这样的异步协程机制去支撑Mutex，属于实验性尝试，可靠性待验证。Linux以及ArceOS的main分支都未采取这种激进的做法。

WaitQueue(Completion)

差异：ArceOS的WaitQueue是一个同步原语，在Linux中与之最接近的是Completion，二者接口的各个方法基本上是一一对应的。差异主要体现在具体内部实现上，ArceOS dev分支的WaitQueue也是基于第三方的crate *event_listener* 实现，这一点与Linux乃至ArceOS main分支的实现都不同。



Linux和ArceOS main分支在实现WaitQ(Completion)时，只是针对一套基于线程的状态机进行任务状态管理，并且让任务在运行队列和等待队列之间迁移。图中的蓝色部分。

但是ArceOS dev分支在上述基础上又引入了基于协程的调度的状态机(图中的红色部分)，大致流程是：Waiting Task调用wait方法时，将会通过block_on方法把本线程变成一个协程异步的运行时，被唤醒时进入协程的Ready状态，未被唤醒时处于Pending状态并通过线程状态机把自己置于阻塞睡眠状态；Notifier Task在调用notify时，需要设置协程状态到Ready并且通过注册的Waker去唤醒waiting task本身。这套实现需要协调两套状态机和调度机制，配合完成wait/notify的同步。

影响：ArceOS dev分支对WaitQueue的实现比较复杂并且是实验性质，可靠性待验证。

Migration&&Affinity

导致任务在CPU间迁移的主要触发因素有两个：

1. 主动设置任务的affinity（主动方式）

   对于Linux，它的Syscall *sched_setaffinity*可以针对任何的目标任务改变其affinity，进而导致其在下次调度时发生CPU迁移；但是ArceOS仅提供了一个改变自己的affinity的api *set_current_affinity*，因此不存在被其它任务强制改变运行CPU的可能，从这方面需要关抢占的必须情况也就比较少。

2. 任务在CPU间的自动负载均衡策略（被动方式）

   对于Linux，在所有发生任务schedule的时机点上，都有触发自动负载均衡的可能；但是对于ArceOS的dev分支，只有两个点：在spawn时和从blocked状态被唤醒时；对于ArceOS的main分支，则只有spawn时。

影响：关抢占的其中一个目的是防止发生CPU迁移，对于ArceOS的dev分支来说，只要注意避开上述三种情况，可以减少关抢占。

Resched(Schedule)

关于核心的任务间切换机制(以switch_to为中心)和任务状态机的状态切换，Linux和ArceOS在大致原理上类似，但是细节上差异较大。总体上看，ArceOS要简单很多，Linux的经验不能照搬，但应该是有好的参考价值。上一轮对二者的比较分析的程度不够，需要再深入分析一次，重点是在Linux方面我没有看懂的部分。

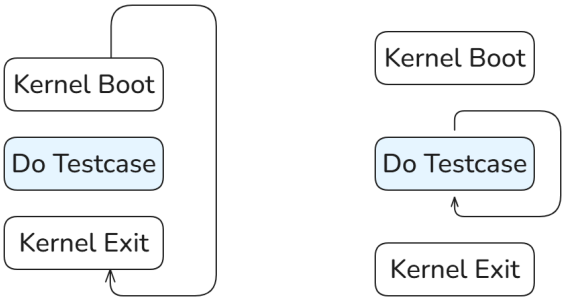
测试设置

目前的多核并发问题都是**概率性**发生，需要找到尽量稳定和高频触发的条件，总结以下四点：

1. 多核数量SMP=8

   基于8核测试触发问题的概率明显高于4核和2核。但不必增加更多的核数，既没有必要，也增加了分析复杂度。

2. 对每个Testcase重复测试的模式是：LOOP（Boot + Test + Exit）



需要对Testcase进行多轮重复测试以触发问题，图中左边的方式触发频率远高于右边。

3. 尽量模拟器环境测试

   多核问题发生有随机性。但是模拟器环境的随机性比硬件环境的随机性低，相对容易定位。

4. 调度测试采取Round Robin

   调度策略RR触发问题的频率远高于CFS策略。

测试结果

测试用例都是基于现有的arceos-apps/task下各个应用，个别有改造。

SMP下触发并发异常的概率如下：

affinity	irq	parallel	priority	sleep	wait_queue	yield
< %1	> 5%	> 5%	< %5	< %5	**> 15%**	< %5

频率最高的是wait_queue，其次是irq和parallel。

主要问题的分析

目前主要问题集中在对**event_listener**这个crate的使用和封装上，尤其是在Blocked的任务被唤醒的环节上无法唤醒，导致系统卡死。具体的问题中，有两个比较突出：

1. 协程调度和传统调度并存，难以同步协调

   在多核情况下，Waiting Task和Notifier Task经常位于不同的CPU上，是真正的并行，双方会构成各种各样的时序组合。测试中发现的典型情况之一：

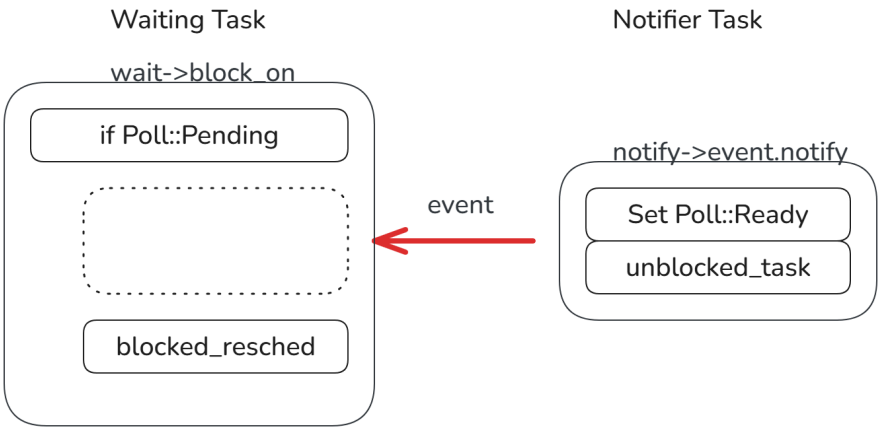
Waiting Task

```
loop {
    woke.store(false, Ordering::Release);
    match fut.as_mut().poll(&mut cx) {
        Poll::Pending => {
            if !woke.load(Ordering::Acquire) {
                [*** 空隙 ***]
                current_run_queue::<NoPreemptIrqSave>().blocked_resched();
            } else {
                // Immediately woken
                crate::yield_now();
            }
        }
        Poll::Ready(output) => {
            ... ...
        }
    }
}
```

Notifier Task

Set Poll::Ready

```
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        self.woke.store(true, Ordering::Release);
        select_run_queue::<NoPreemptIrqSave>(&task).unblock_task(task, false);
    }
}
```



Waiting Task调用poll得到Pending，认为未被唤醒，正常会阻塞自己进入睡眠；但是Notifer Task恰好在这两个动作之间执行了唤醒，那么就是唤醒后立刻又被阻塞睡眠了。Notifer Task已经完成了自己的唤醒工作，通常不能期望它重复该动作，但是Waiting Task却已经进入到睡眠，没有机会重新调用poll来再次检查。当然，如果有机会被唤醒再次检查，**多数情况**下会发现协程状态变成Ready，这样能够正常进行下去。

2. 第三方crate event_listener在多核并行情况下运行不稳定

   等待/唤醒机制是在对crate event_listener封装的基础上实现的，但是在SMP情况下，存在一定的概率，针对目标任务的唤醒notify动作已经执行，但是Poll的状态没有变成Ready；又或者在没有对某Waiting Task唤醒的情况下，该Task注册的Waker被其它线程调用，被提前唤醒引起意外的并发问题。

   一方面可能是crate event_listener本身存在bug，另一方面也可能是ArceOS dev对它的封装使用方式不合适。目前还无法判断具体哪种原因。

测试分支

在git@github.com:shilei-masclouds/org_arceos.git的**dev-smp**可以复现研究上述问题。该分支基于arceos-org的dev分支(commit **16096568f5a**)。

预设断言

目前开始在当前代码的基础上，参考Linux的方式，尽量增加断言，以提高发现和定位问题的效率。

例如，在中断上下文和自旋锁的开始结束处，标记atomic context；同时在resched()核心函数中检查，是否处于atomic context中，以此发现违例的情况。